

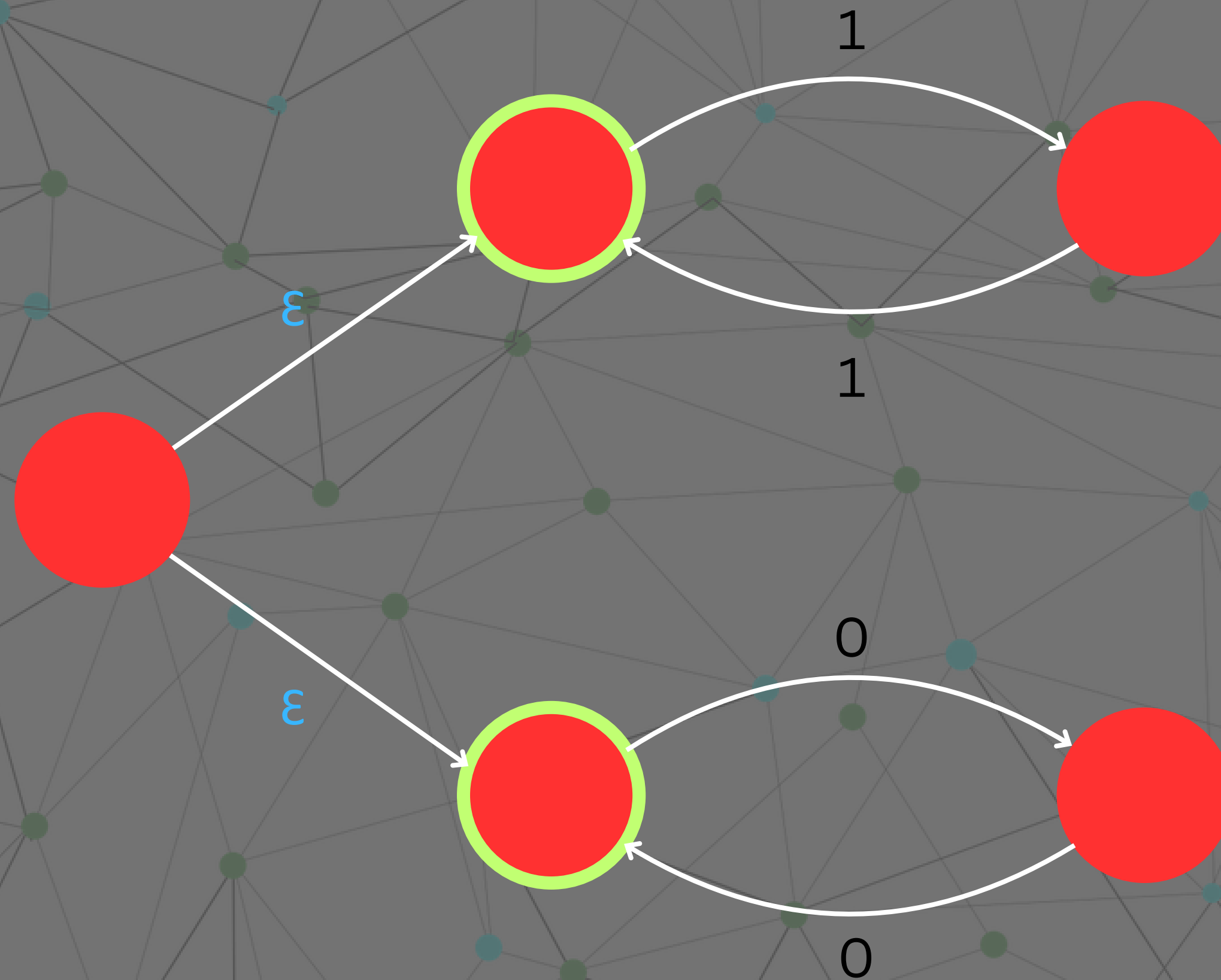
Compiladores



Clase 1/10/2026

Walter A Segura
Barboza
2023072838

Recordemos



Teníamos autómatas
con transiciones
épsilon

Podemos avanzar por
el autómata sin gastar
un símbolo

¿Pero qué es realmente una
transición épsilon?

Propiedades

Si usamos una transición ϵ podemos saltar al estado sin gastar algún símbolo

Podemos hacer eso incluso *ya acabada la hilera*

Es opcional usarlas

Tienen naturaleza *no determinística*



Un autómata con estas transiciones ϵ se conoce como **NFA- ϵ**

*¿y formalmente como
definimos esto?*

Definición normal

Un Autómata No Determinístico de Estados Finitos, con transiciones ϵ (NFA- ϵ) es un quinteto $M = (Q, \Sigma, \delta, q_0, F)$ donde:

Q es un conjunto finito de **estados**

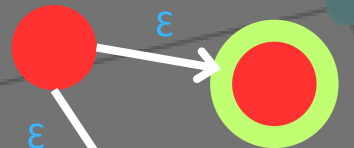
Σ es un **alfabeto**

$\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow P(Q)$ es la **función de transición**

$q_0 \in Q$ es el **estado inicial**

$F \subseteq Q$ conjunto de **estados de aceptación**

puedo ver símbolos
de Σ y a ϵ



Teorema

Sea $M = (Q, \Sigma, \delta, q_0, F)$ un NFA- ϵ . Siempre existe un NFA $M' = (P(Q), \Sigma, \delta', q_0, F')$ que reconoce exactamente el mismo lenguaje



Básicamente que si
tenemos un NFA- ϵ , existe
un NFA equivalente



Colorario

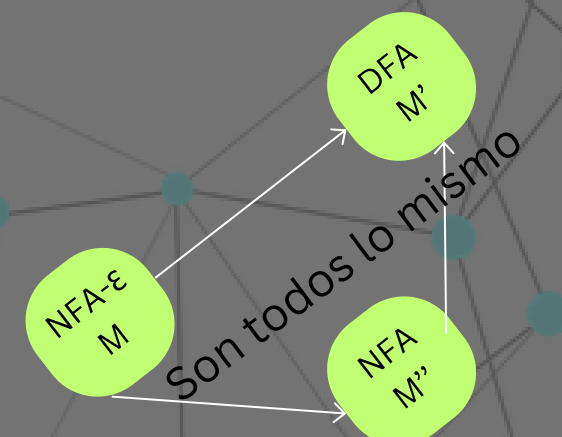
Colorario:
conclusión lógica
“obvia”, que no
requiere mucho
esfuerzo a partir de
un teorema



Con $M = (Q, \Sigma, \delta, q_0, F)$ un NFA- ϵ . Siempre existe un DFA, $M' = (P(Q), \Sigma, \delta', q_0, F')$ que reconoce exactamente el mismo lenguaje que M .



Same thing



Cierre - ϵ

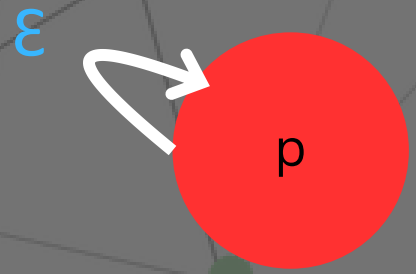
También
llamado
Alcance - ϵ



Función que recibe un estado p de Q y regresa un subconjunto de Q

Este subconjunto son todos los estados alcanzables desde p sin consumir ningún símbolo de la entrada, ósea, estados a los que podemos llegar usando transiciones ϵ

Recordemos que siempre hay una transición ϵ al mismo estado, entonces nunca es conjunto vacío, a lo menos es $\{p\}$



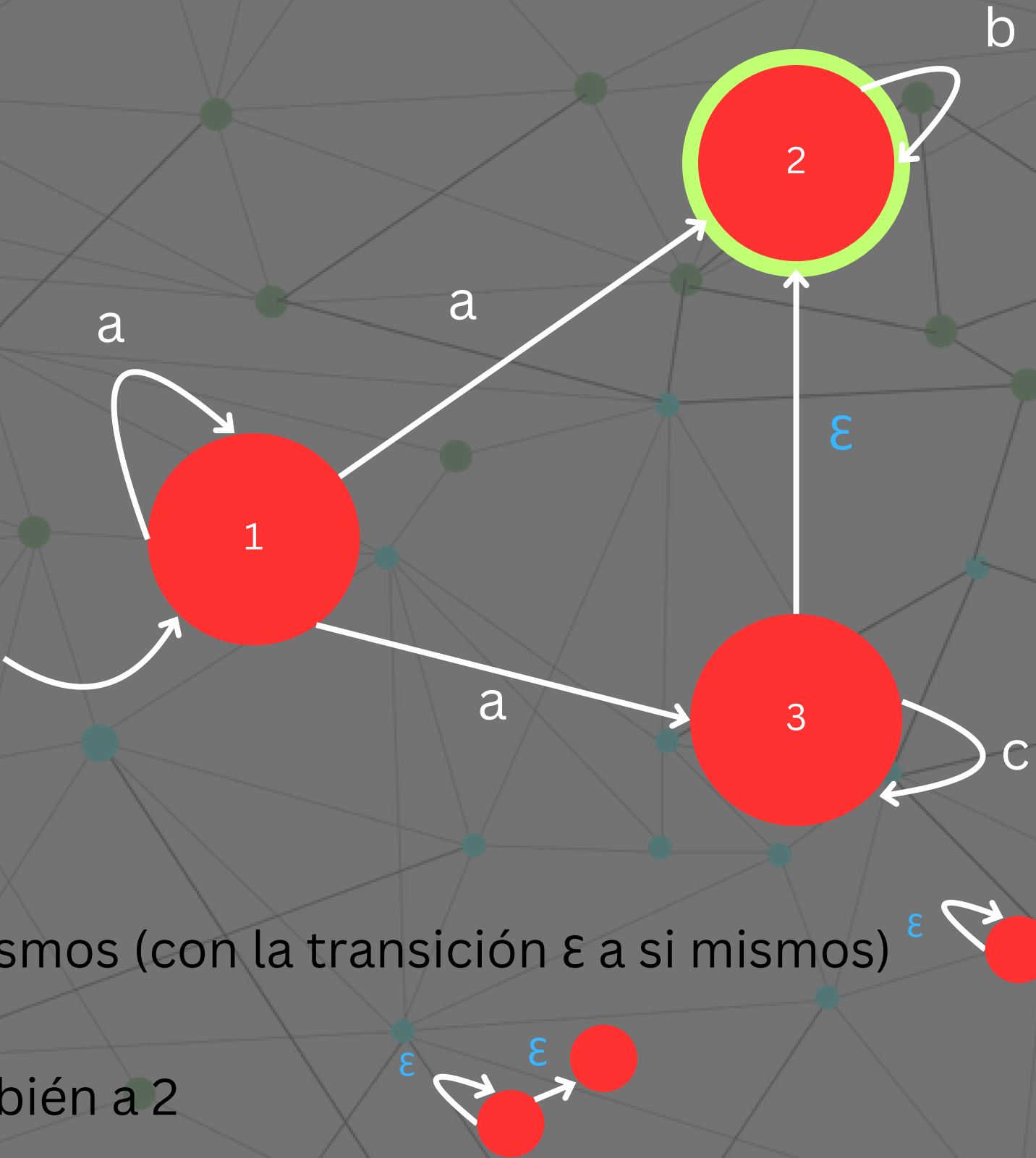
Empecemos a usar este cierre de ahora en adelante para nuestros cálculos, en lugar de usar p , usaremos el cierre- ϵ de p

Cierre - ϵ

Por ejemplo

Estado (p)	Cierre- ϵ
1	{1}
2	{2}
3	{2,3}

Aquí tenemos todos los estados a los que puedo llegar sin gastar símbolos desde p



En el caso del 1 y 2, solo podemos ir a ellos mismos (con la transición ϵ a si mismos)

En el caso del 3 no solo puedo ir a 3, pero también a 2

Convirtiendo NFA- ϵ en DFA

Similar a lo que ya hemos hecho con NFA a DFA

El alfabeto es el mismo en ambas máquinas

Los estados de M' (DFA) son subconjuntos del conjunto Q de M (NFA- ϵ)

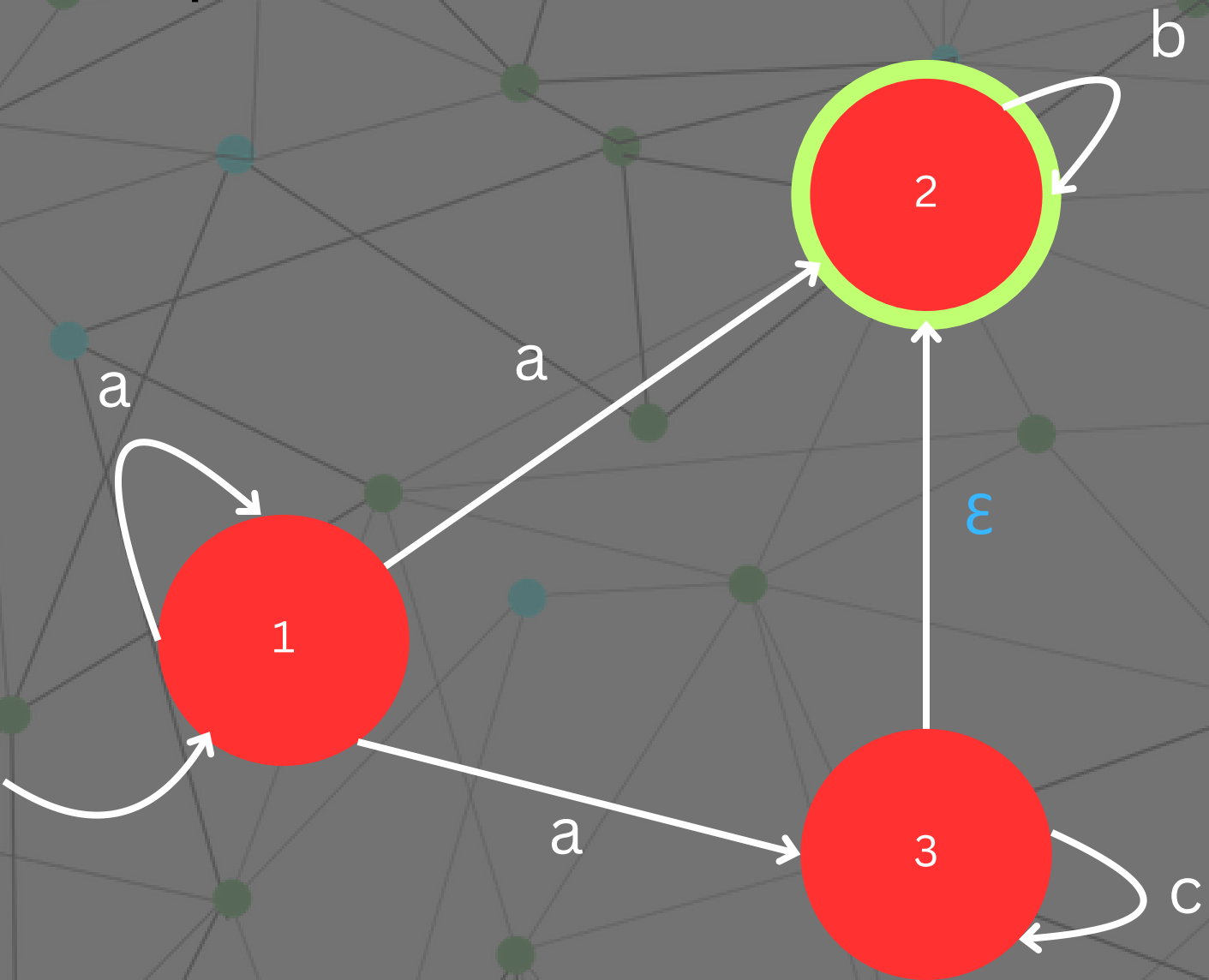
F' es un conjunto de subconjuntos del conjunto Q de M



Como primer paso calculamos el Cierre- ϵ de todos los estados de Q

Convirtiendo NFA- ϵ en DFA

Tomemos esta
máquina



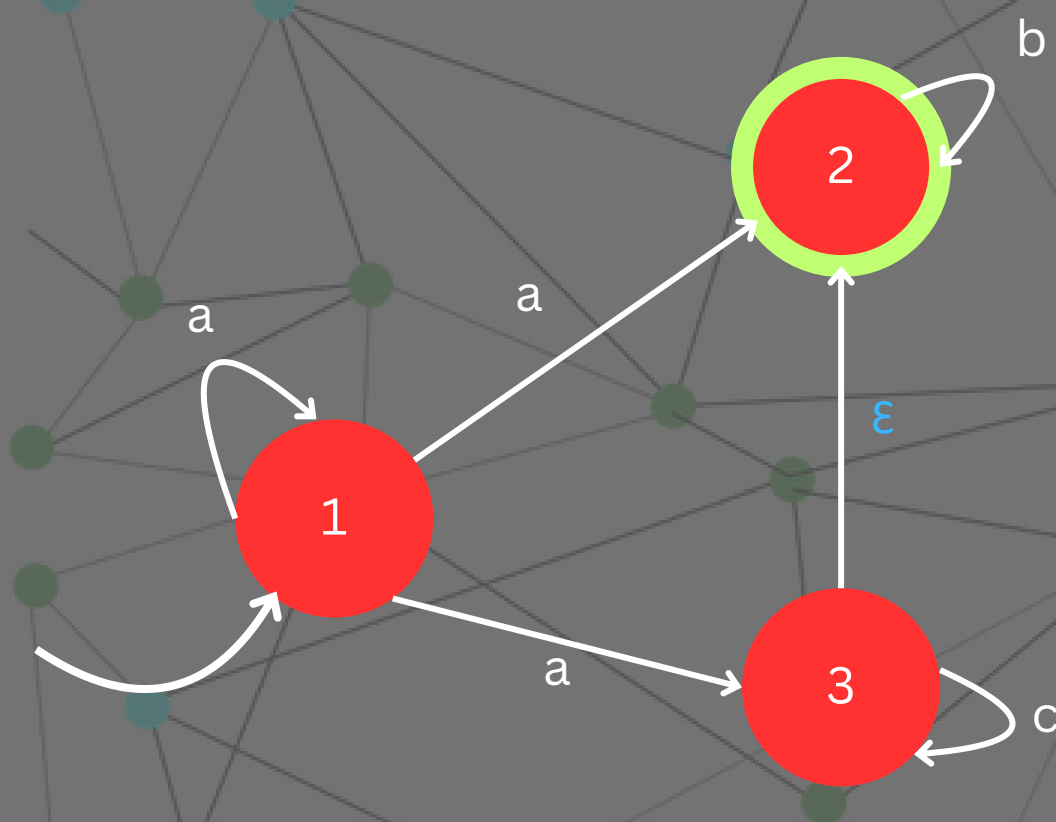
Como vimos antes, sus Cierres
eran

Estado	Cierre- ϵ
1	{1}
2	{2}
3	{2,3}

Con esto hecho, ahora simplemente empezamos a
hacer nuestro procedimiento de siempre, tomando
en cuenta los cierres para cada elemento

Convirtiendo NFA- ϵ en DFA

Como ven es el mismo procedimiento de antes



Estado	Cierre- ϵ
1	{1}
2	{2}
3	{2,3}

a	b	c	
{1,2,3}	$\emptyset$	$\emptyset$	{1}
{1,2,3}	{2}	{2,3}	{1,2,3}
$\emptyset$	{2}	$\emptyset$	{2}
$\emptyset$	{2}	{2,3}	{2,3}



Notar como aquí el conjunto es {2,3} debido a que desde 3 podemos ir a 2 usando a ϵ , sin importar que el símbolo sea c.

Convirtiendo NFA- ϵ en DFA

{1}

{1,2,3}

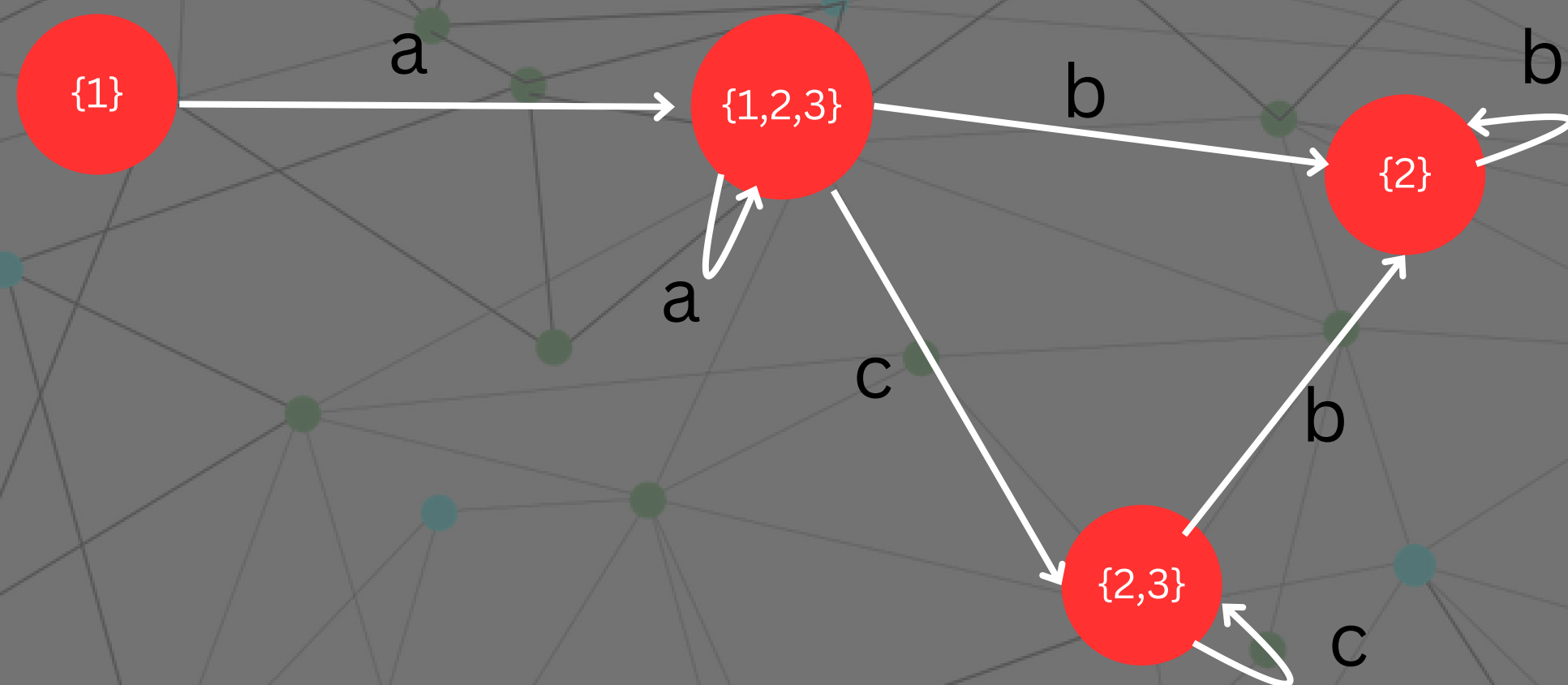
{2}

{3}

	a	b	c
{1}	{1,2,3}	$\emptyset$	$\emptyset$
{1,2,3}	{1,2,3}	{2}	{2,3}
{2}	$\emptyset$	{2}	$\emptyset$
{2,3}	$\emptyset$	{2}	{2,3}

Ponemos el estado de la primera columna de cada fila

Convirtiendo NFA-ε en DFA



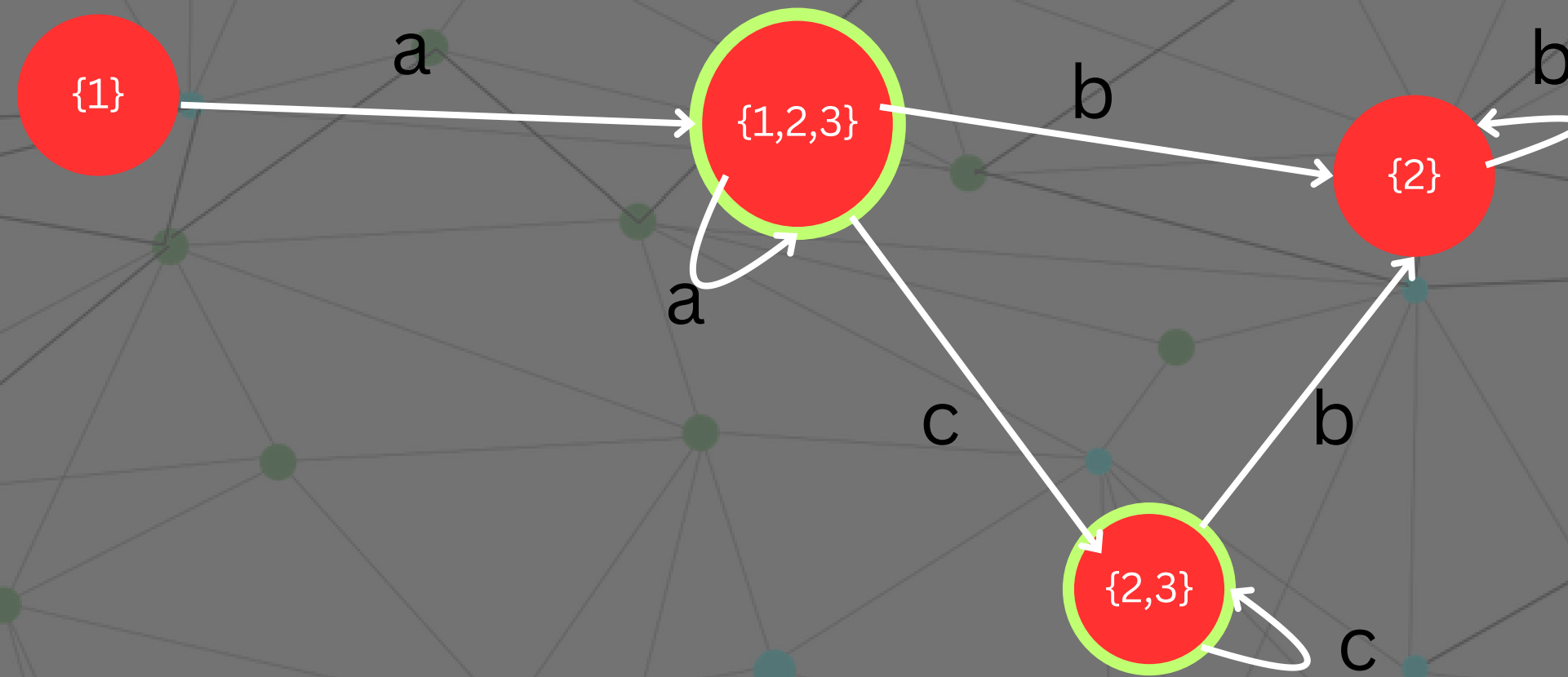
	a	b	c
$\{1\}$	$\{1,2,3\}$	$\emptyset$	$\emptyset$
$\{1,2,3\}$	$\{1,2,3\}$	$\{2\}$	$\{2,3\}$
$\{2\}$	$\emptyset$	$\{2\}$	$\emptyset$
$\{2,3\}$	$\emptyset$	$\{2\}$	$\{2,3\}$

Las transiciones de cada uno según las demás columnas



Convirtiendo NFA- ϵ en DFA

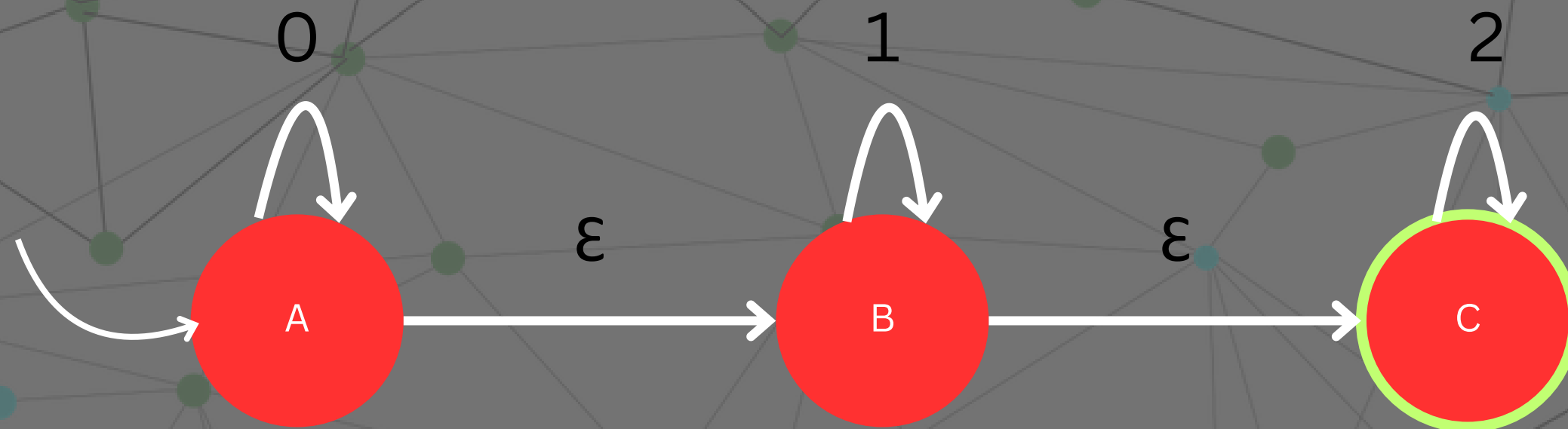
Flex modela
NFA- ϵ y
luego los
convierte
en DFA



Y recordamos que todos los estados que tengan en su conjunto algún estado de aceptación de la máquina original, pasan a ser estados de aceptación en esta

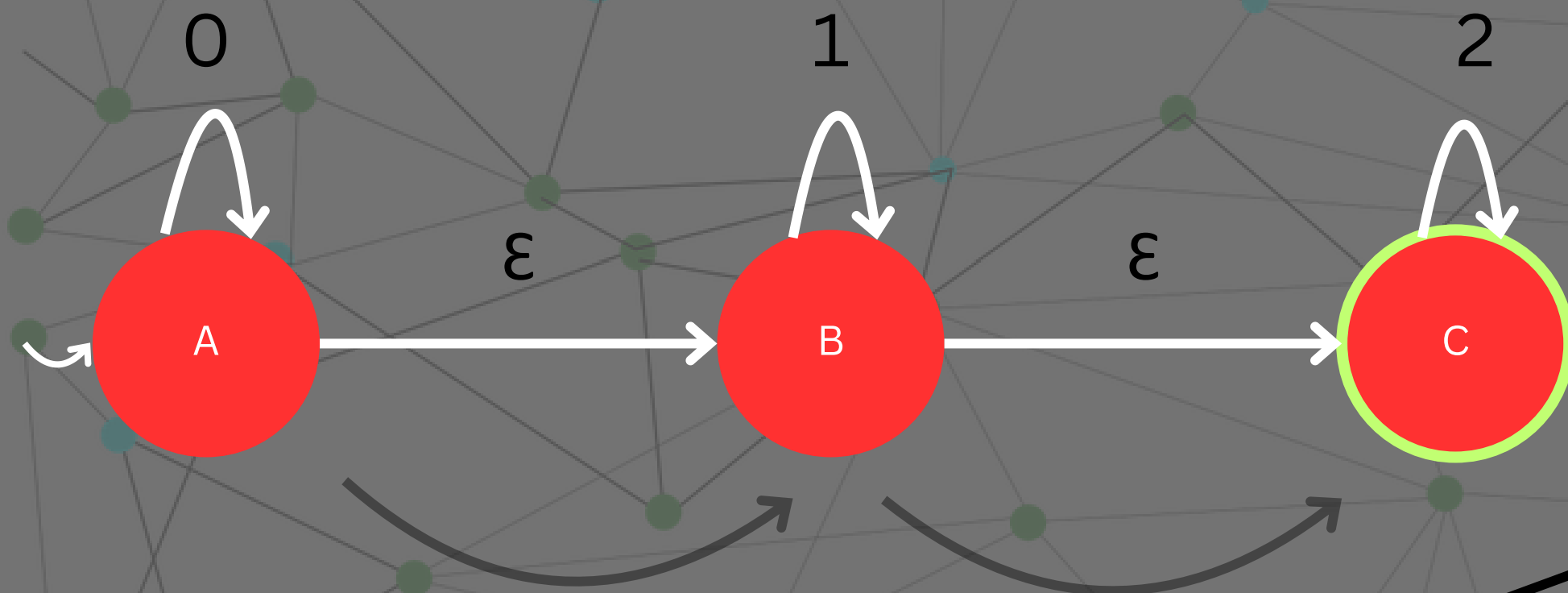
Más ejemplos

Ejemplo 2



Primero lo primero, sacamos Cierre - ϵ

Más ejemplos

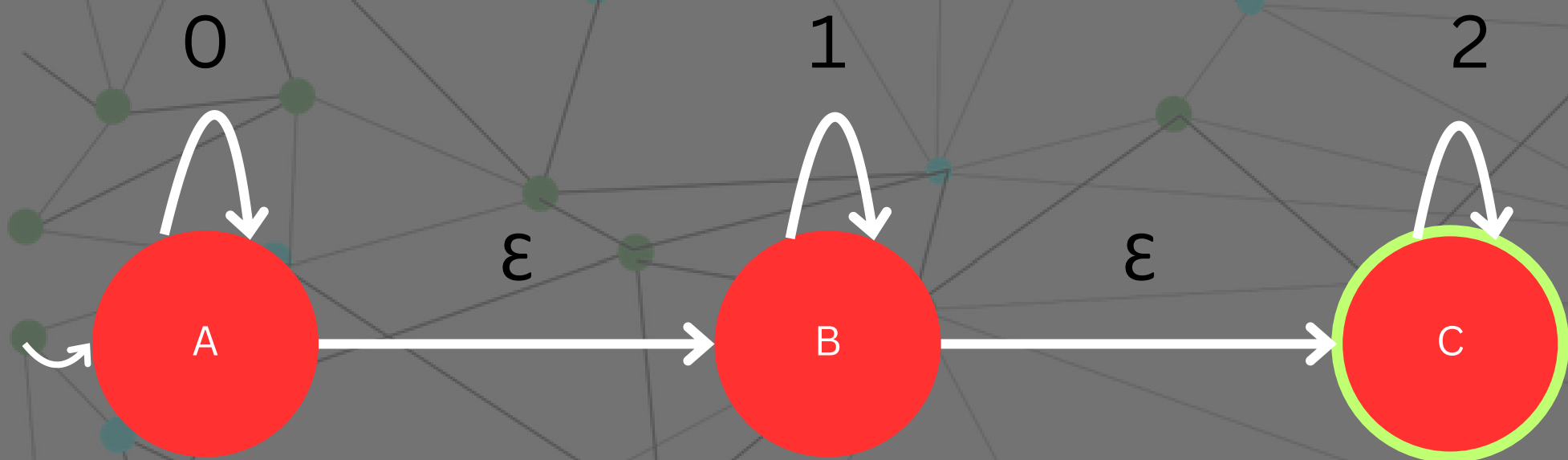


Estado	Cierre- ϵ
A	{A,B,C}
B	{B,C}
C	{C}

Noten como en el caso de la A, pude avanzar a B con épsilon, y desde B pude avanzar a C con épsilon, entonces pongo los dos estados en el Cierre



Más ejemplos

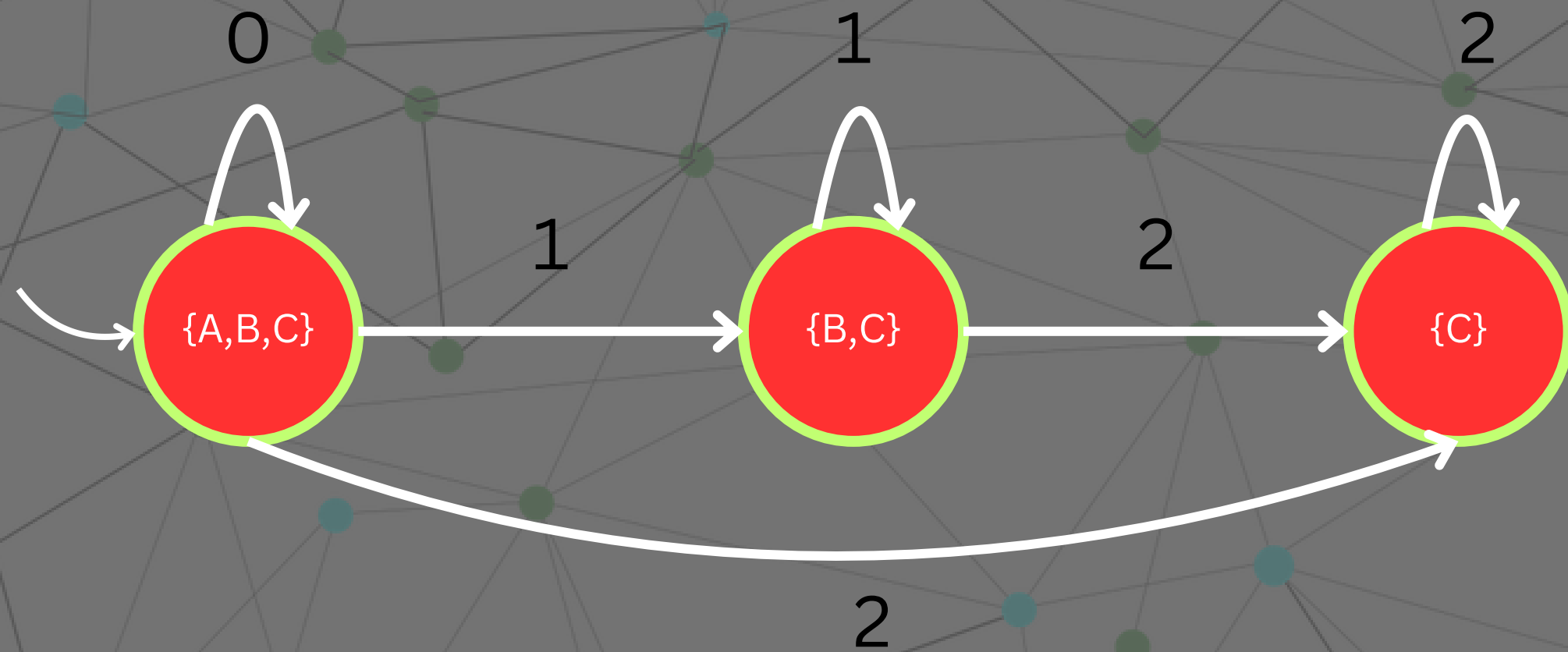


Estado	Cierre- ϵ
A	{A,B,C}
B	{B,C}
C	{C}

	0	1	2
{A,B,C}	{B,C}	{A,B,C}	{C}
$\emptyset$	{B,C}	{B,C}	{C}
$\emptyset$	$\emptyset$	{C}	{C}

Más ejemplos

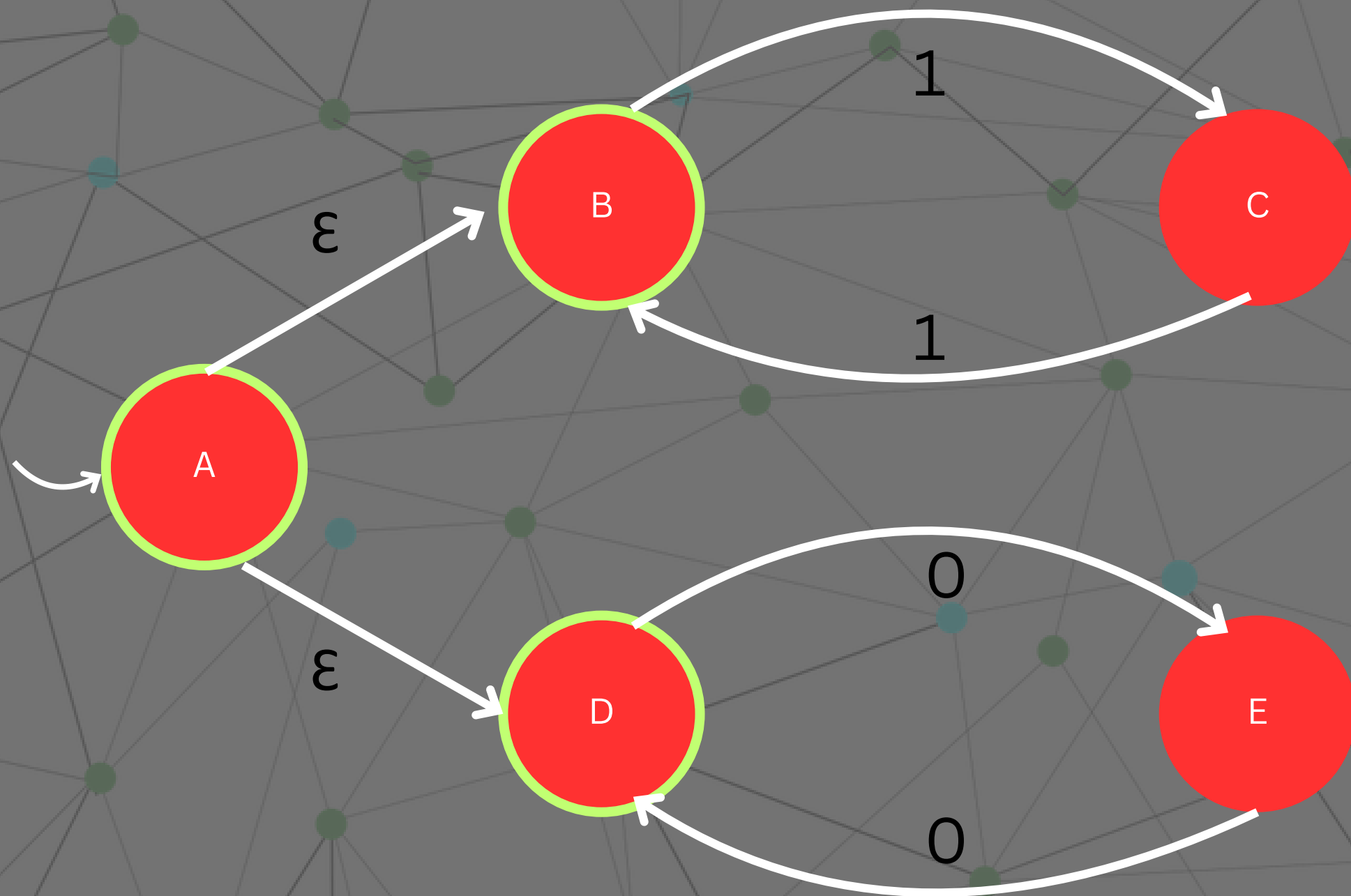
Friendly reminder: si un autómata no puede transaccionar el siguiente símbolo entonces se rechaza la hilera, no es que este autómata este aceptando todas



	0	1	2
$\{A,B,C\}$	$\{A,B,C\}$	$\{B,C\}$	$\{C\}$
$\{B,C\}$	$\emptyset$	$\{B,C\}$	$\{C\}$
$\{C\}$	$\emptyset$	$\emptyset$	$\{C\}$

Más ejemplos

Ejemplo 3

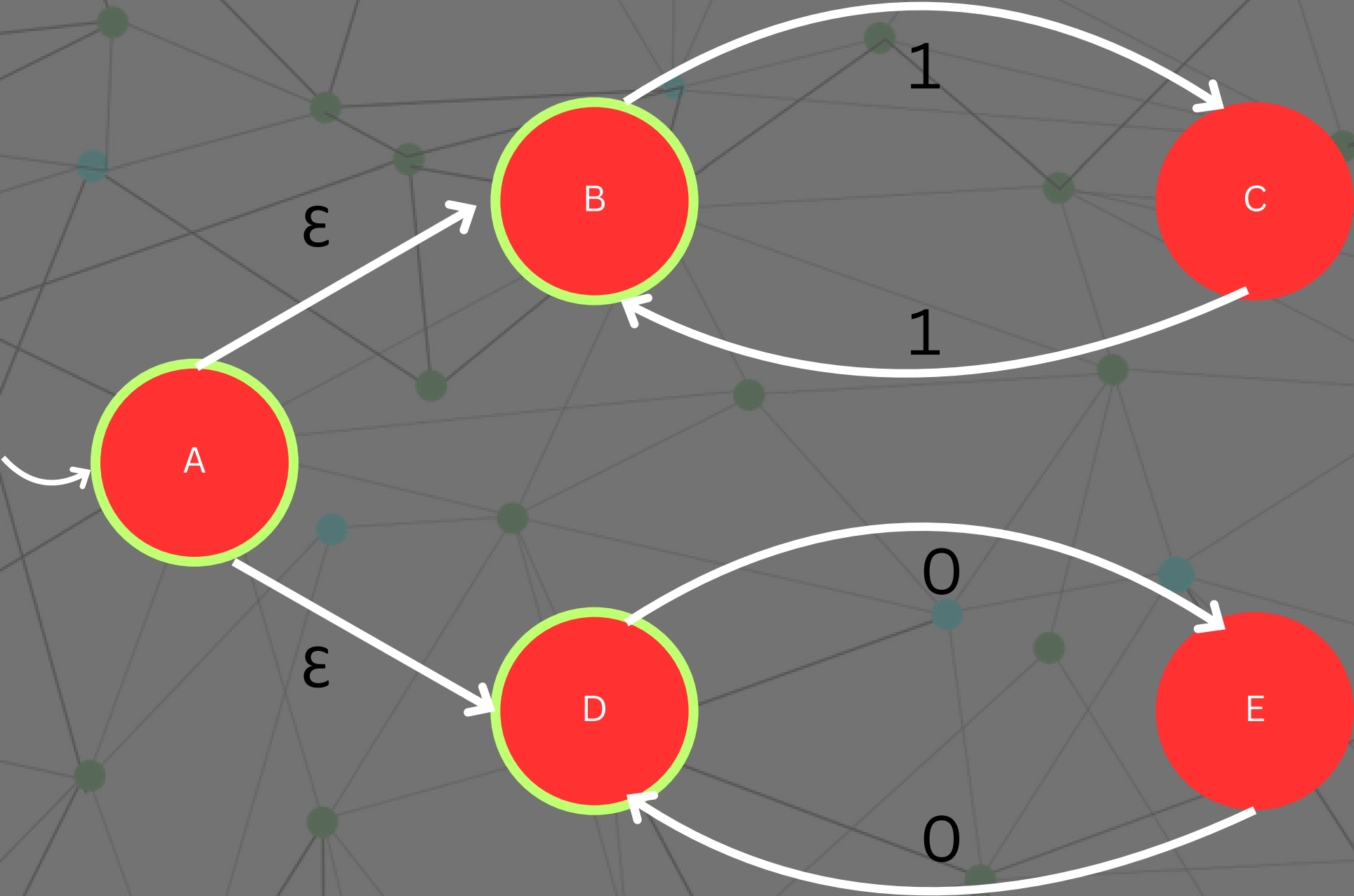


Primer paso, crear la tabla de Cierres!



Más ejemplos

Ejemplo 3



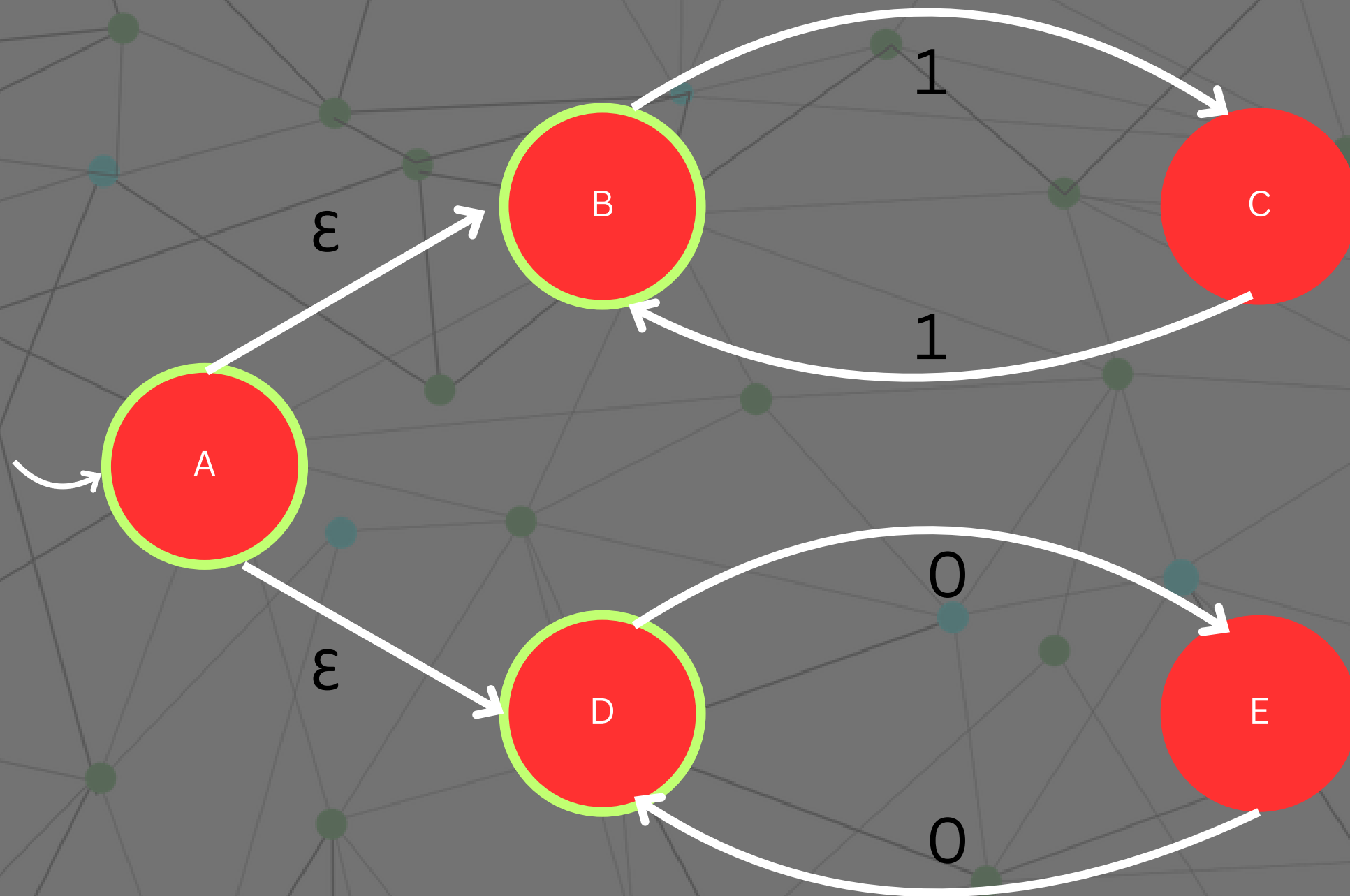
Estado	Cierre-ε
A	{A,B,D}
B	{B}
C	{C}
D	{D}
E	{E}

Empecemos con este



Más ejemplos

Ejemplo 3



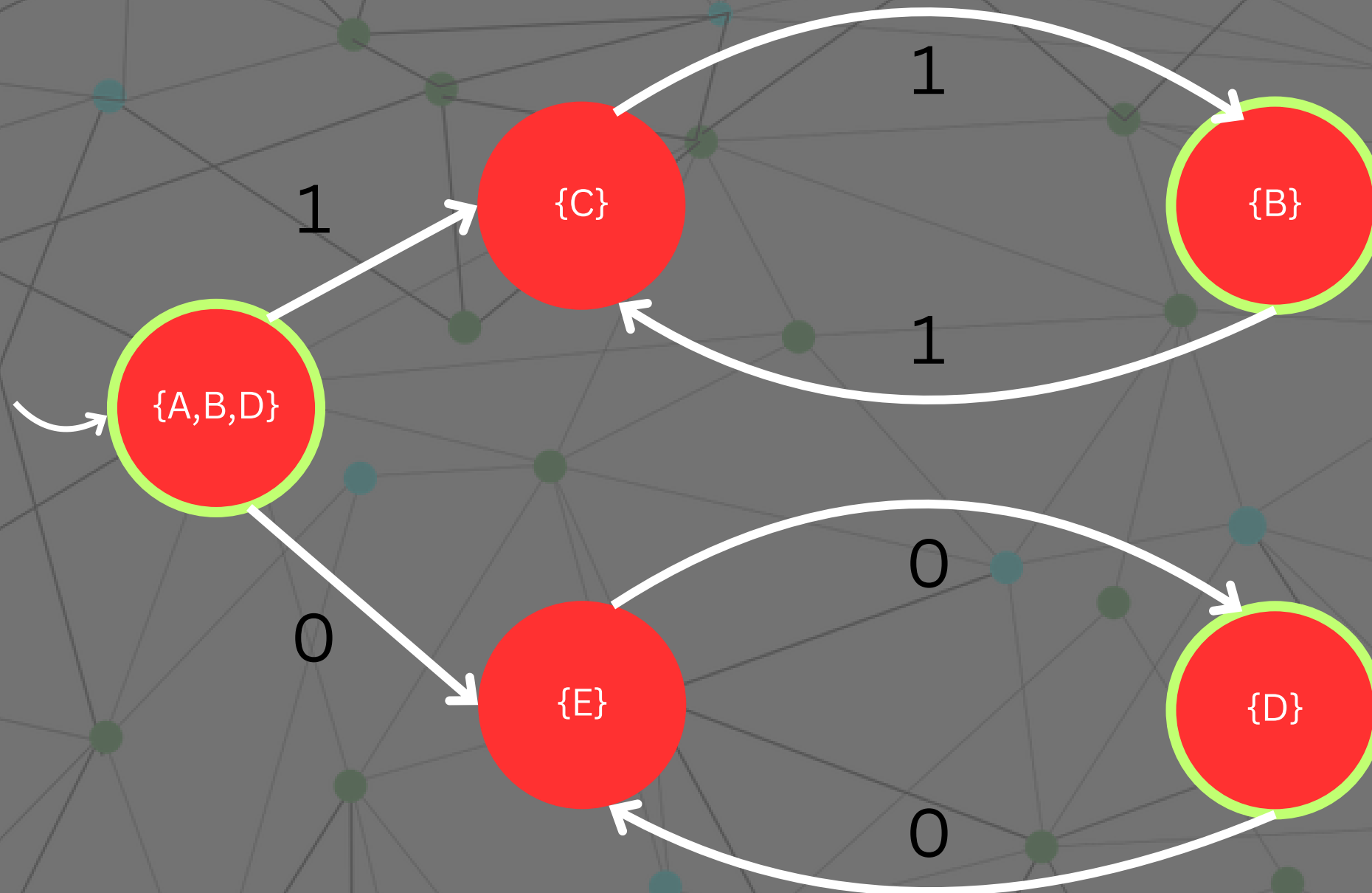
0	1	
{E}	{C}	{A,B,D}
{D}	∅	{E}
∅	{B}	{C}
∅	{C}	{B}
{E}	∅	{D}

Más ejemplos

Parece que solo le dimos vuelta



Ejemplo 3



	0	1
{A,B,D}	{E}	{C}
{E}	{D}	∅
{C}	∅	{B}
{B}	∅	{C}
{D}	{E}	∅

Lenguajes Regulares

Definición

Un lenguaje $\mathcal{L}$ es regular si y solo si existe un DFA que lo reconozca exactamente (acepte todas las hileras de $\mathcal{L}$ y rechace cualquiera que no sea miembro de $\mathcal{L}$)

Básicamente un lenguaje regular es aquel que tiene un DFA que lo reconozca y viceversa

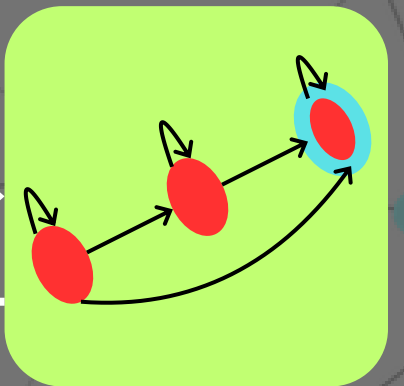
si y solo si



Implicación es bidireccional, ambos comparten el mismo valor de verdad. Ambos son verdaderos, o ambos son falsos



$\mathcal{L}$ es regular



Lenguajes Regulares

Sabemos que todo NFA o NFA- ϵ tiene un DFA equivalente, entonces basta con que exista uno de estos para que el lenguaje sea regular

Estos **no** son más poderosos que un DFA

Entonces ¿como demostramos que un lenguaje es regular?

¡Pues es lo que hemos estado haciendo! **Diseñar un DFA que lo reconozca**

Entonces ¿como demostramos que un lenguaje NO es regular?

Demostrar que **no existe ningún DFA que lo reconozca**

Pumping Lemma

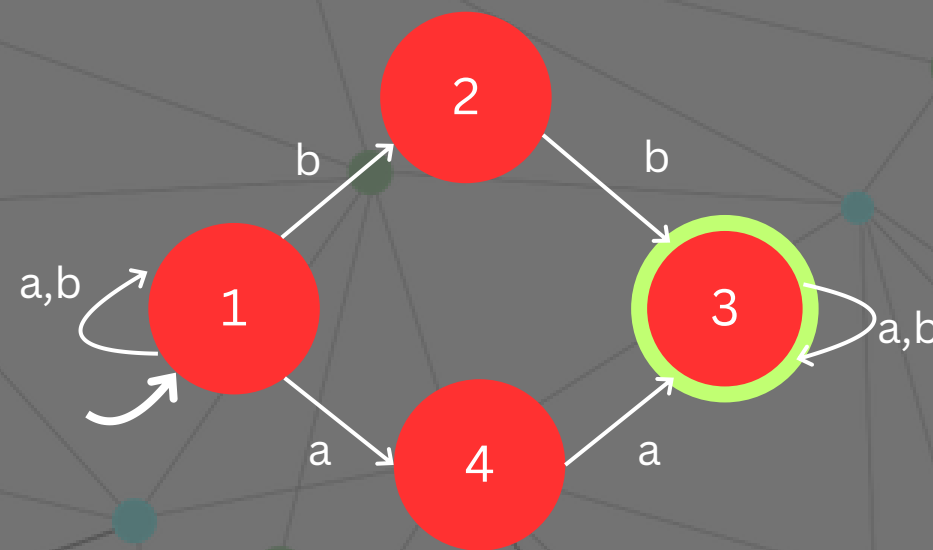
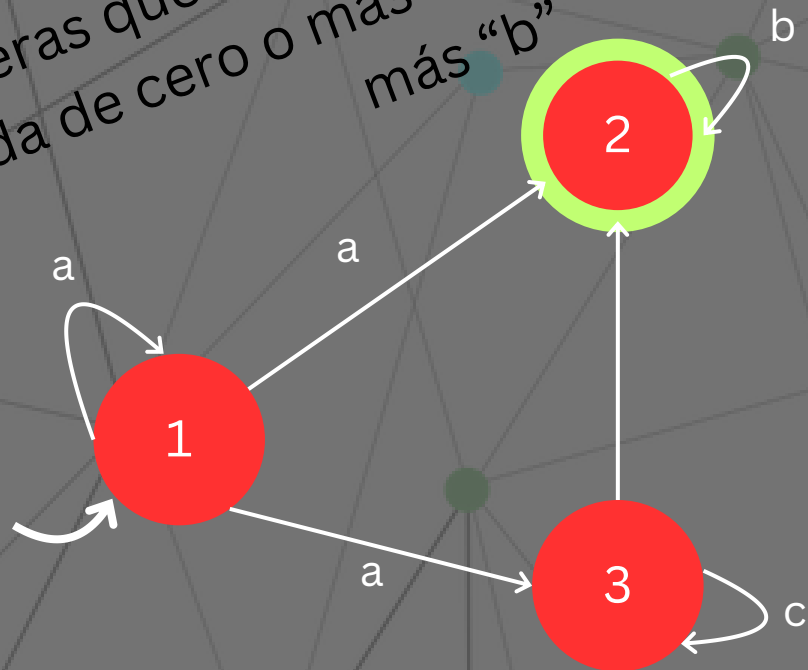


Lenguajes Regulares - Ejemplos

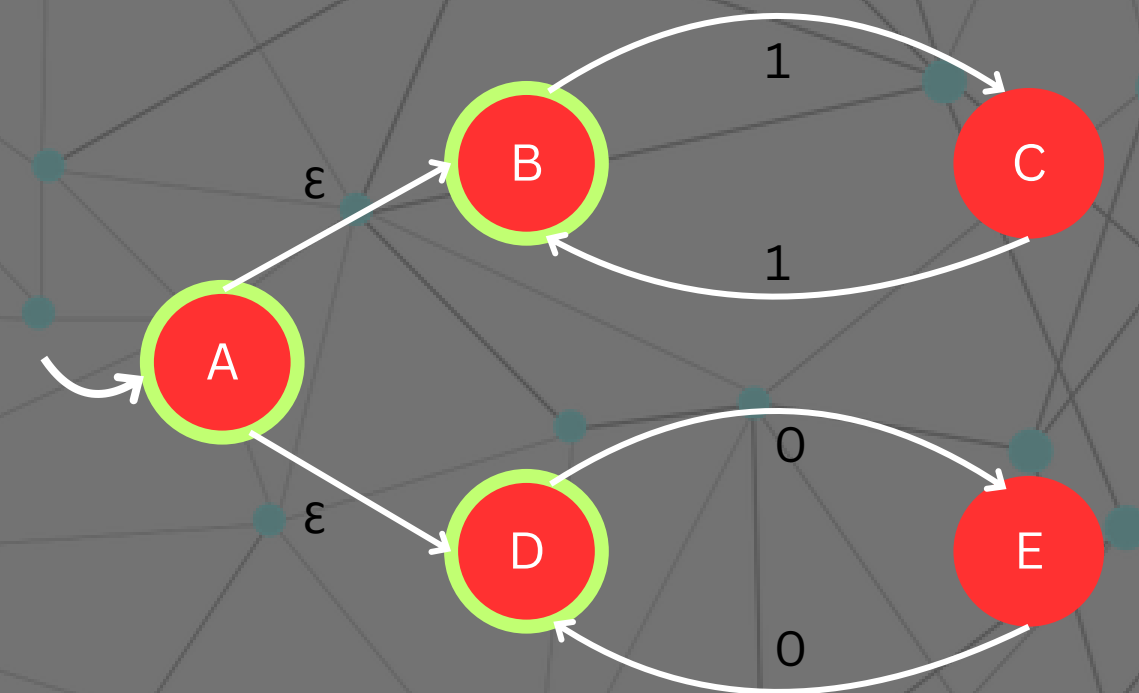
Básicamente todos los lenguajes con los que hemos trabajado hasta ahorita son ejemplos de lenguajes regulares, ya que para cada uno logramos hacer un DFA (NFA, NFA- ϵ) que lo reconociera

$\mathcal{N}$ = hileras con la subhilera "aa" o "bb"

$\mathcal{L}$ = hileras que empiezan con una serie de "a", seguida de cero o más "c" y terminada con 0 o más "b"



$\mathcal{M}$ = hileras con longitud par con un único tipo de símbolo (0,1)



**Todos
regulares**

Cierre en conjuntos

Un conjunto X puede ser cerrado con respecto a una operación ϕ

Esto significa que, si X es cerrado respecto a ϕ , operar elementos de X con la operación ϕ , el resultado siempre será parte de X también

Por ejemplo el conjunto de los números naturales es cerrado respecto a la suma (+). Esto significa que si sumo dos naturales, sin importar cuales sean, el resultado de la suma es otro número natural

Pero no son cerrados respecto a la resta, yo puedo restar dos naturales, 4 y 10 por ejemplo, y el resultado no es natural (-6)

Entonces, los lenguajes regulares son cerrados respecto a ???



Cierre en Unión de Lenguajes Regulares

Teorema:

Si $\mathcal{L}$ y $\mathcal{M}$ son lenguajes regulares, entonces $\mathcal{L} \cup \mathcal{M}$ también es regular

Los lenguajes regulares son cerrados respecto a la unión

Pero esto es magia, hay que demostrar que es cierto

¿Cómo lo hacemos?

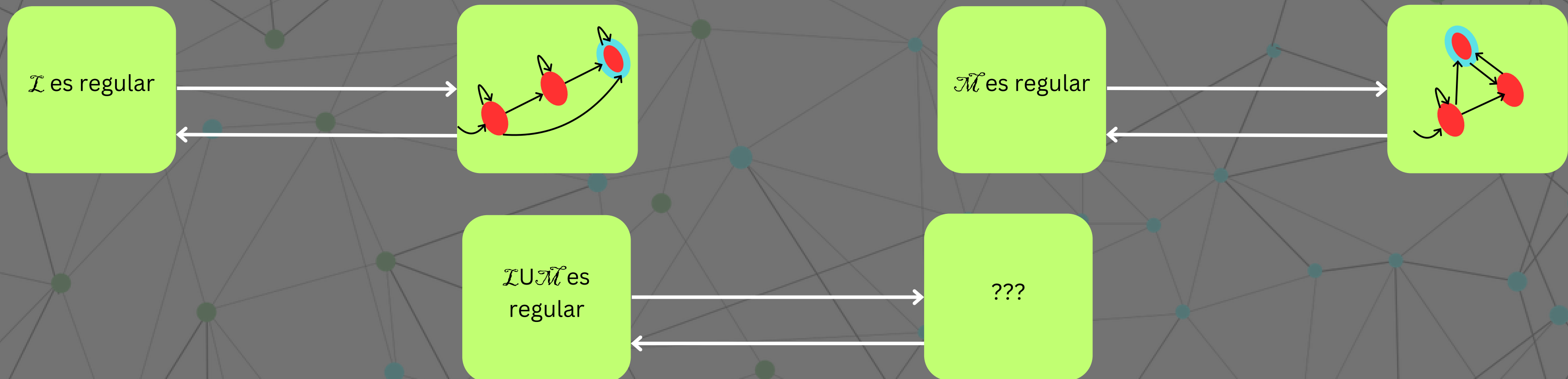


Cierre en Unión de Lenguajes Regulares

Pensemos:

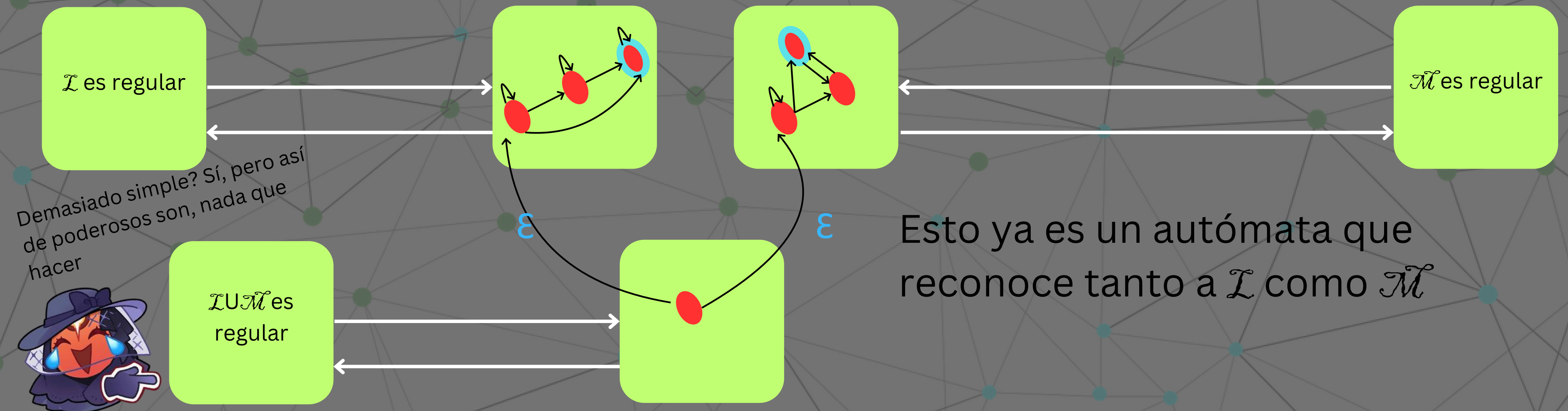
Si $\mathcal{L}$ y $\mathcal{M}$ son regulares, entonces hay un autómata que los reconoce

Y si el resultado de $\mathcal{L} \cup \mathcal{M}$ también es regular, entonces también tiene un autómata que lo reconoce



Cierre en Unión de Lenguajes Regulares

Dejemos de pensar, es tan fácil como unir ambos autómatas



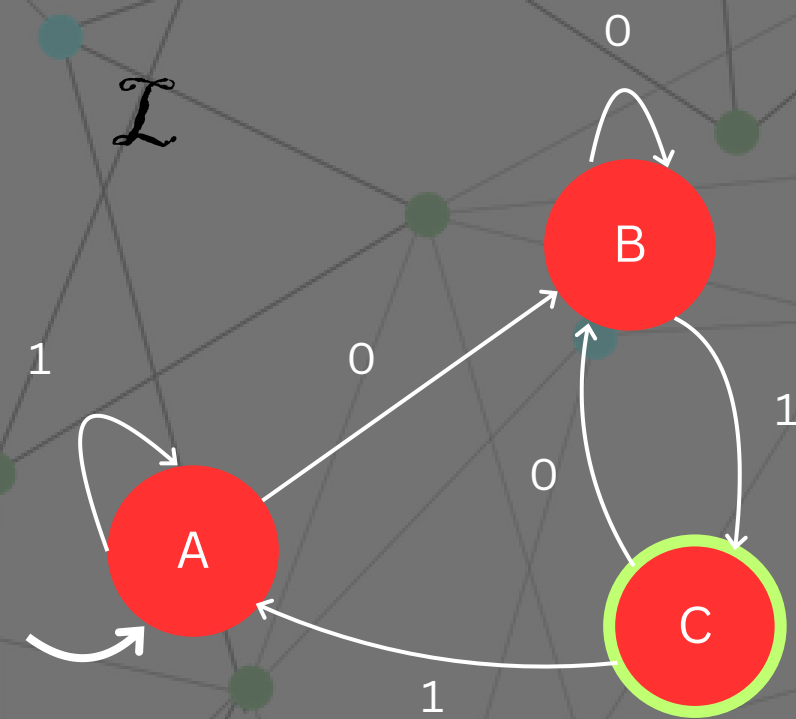
Cierre en Unión de Lenguajes Regulares

Por si no quedo más claro, veamos un ejemplo

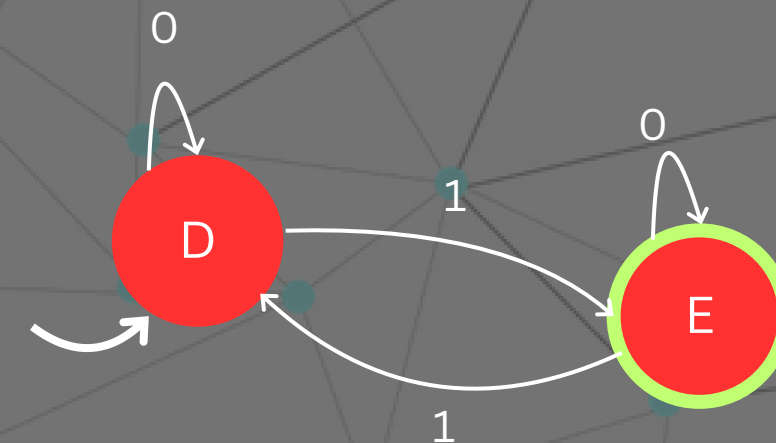
Tenemos dos lenguajes con sus respectivos autómatas

$\mathcal{L}$ = lenguaje sobre $\Sigma = \{0,1\}$ de hileras que terminen en 01

$\mathcal{M}$ = lenguaje sobre $\Sigma = \{0,1\}$ de hileras con un número impar de 1s



$\mathcal{M}$



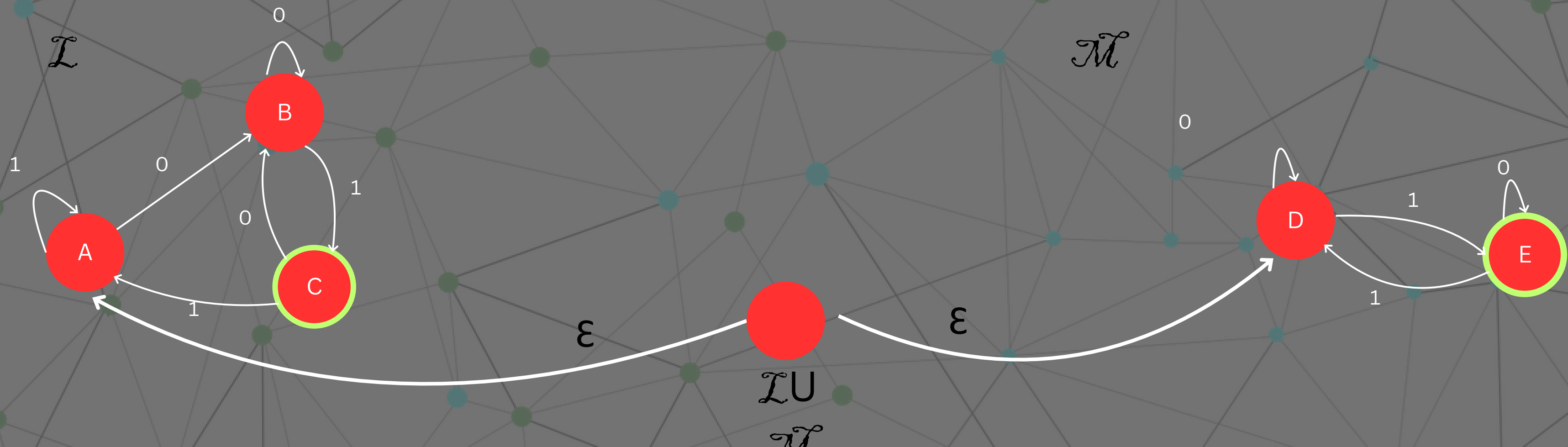
$\mathcal{L} \cup \mathcal{M} =$
???

Cierre en Unión de Lenguajes Regulares

$\mathcal{L}$ = lenguaje sobre $\Sigma = \{0,1\}$ de hileras que terminen en 01

$\mathcal{M}$ = lenguaje sobre $\Sigma = \{0,1\}$ de hileras con un número impar de 1s

Pues unimos las máquinas



Cierre en Complemento de Lenguajes Regulares

Teorema:

Si $\mathcal{L}$ es un lenguaje regular, entonces $\bar{\mathcal{L}}$ también es regular

Los lenguajes regulares son **cerrados** respecto al complemento de lenguajes

¿Otra vez, cómo demostramos esto?



Cierre en Complemento de Lenguajes Regulares

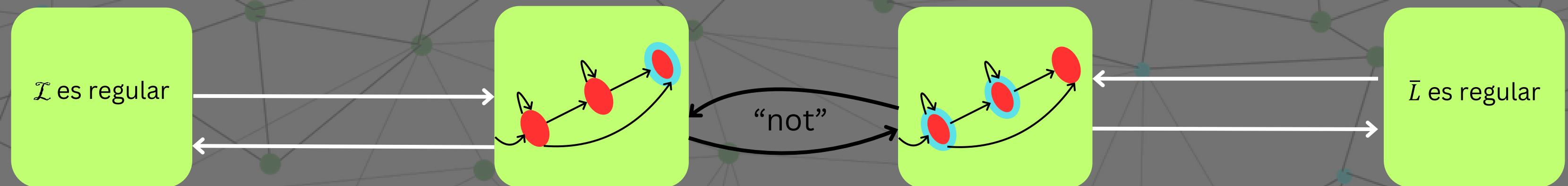
Pensemos

Si $\mathcal{L}$ es un lenguaje regular, entonces existe un autómata que acepta todas las hileras que pertenezcan a este lenguaje, y rechaza las que no.

El complemento de $\mathcal{L}$ son todas las hileras que no pertenecen a $\mathcal{L}$, entonces necesitaríamos un autómata que acepte todas las hileras que no sean de $\mathcal{L}$.

Sin pensarlo mucho, el autómata de $\mathcal{L}$ ya hace este trabajo, simplemente hay que cambiar los estados de aceptación, es como hacerle un complemento al mismo autómata

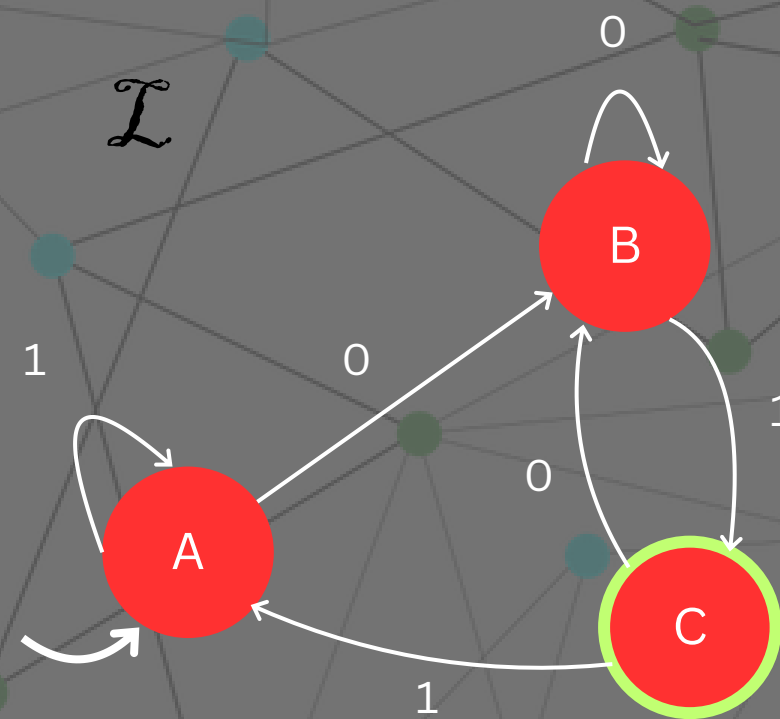
Cierre en Complemento de Lenguajes Regulares



Cierre en Complemento de Lenguajes Regulares

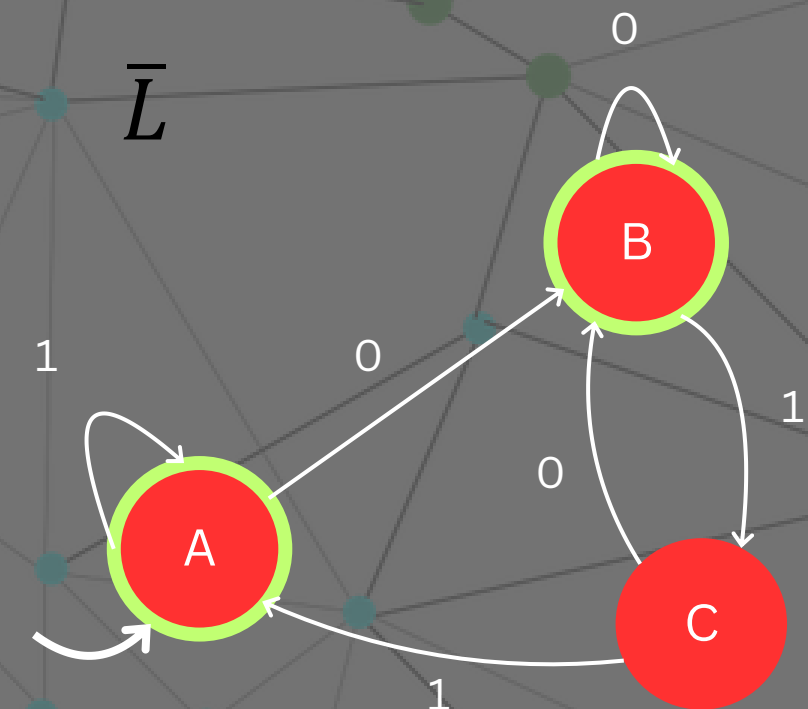
Veamos un ejemplo

$\mathcal{L}$ = lenguaje sobre $\Sigma = \{0,1\}$ de hileras que terminen en 01



$\mathcal{L}$ regular

complemento



Cierre en intersección de Lenguajes Regulares

Teorema:

Si L y M son lenguajes regulares entonces $L \cap M$ también es regular

Los lenguajes regulares son cerrados respecto a la intersección de lenguajes

Abracadabra veamos como funciona esto



Cierre en intersección de Lenguajes Regulares

Esto suena super difícil de imaginar, entonces hagamos algo parecido a elefantes azules

Estamos diciendo que $L \cap M$ el resultado es un lenguaje regular

¿Pero qué es realmente $L \cap M$? Tratemos de transformarlo en algo que ya conocemos

Por discreta sabemos que $L \cap M = \overline{\overline{L} \cap \overline{M}}$. Como el complemento sabemos que es cerrado, entonces no afecta nuestro resultado, pero seguimos teniendo a $\cap$, el cual necesitamos descomponer

Cierre en intersección de Lenguajes Regulares

Si recordamos a De Morgan de discreta, Sabemos que

$$(A \cap B)' = A' \cup B' \quad (A \cup B)' = A' \cap B'$$

Entonces nuestro $\overline{\overline{L} \cap \overline{M}}$, se transforma en $\overline{\overline{L} \cup \overline{M}}$



Y con eso ya lo demostramos, debido a que transformamos $\overline{\overline{L} \cap \overline{M}}$ en $\overline{\overline{L} \cup \overline{M}}$ y $\overline{\overline{L} \cup \overline{M}}$ son puros complementos y unión, y como demostramos antes, estos son cerrados, entonces eso significa que los Lenguajes Regulares también son cerrados respecto a la intersección



Cierre en Concatenación de Lenguajes Regulares

Teorema:

Si L y M son lenguajes regulares, entonces LM también es regular

Los lenguajes regulares son cerrados respecto a la concatenación de lenguajes

¿y este como lo demostramos?

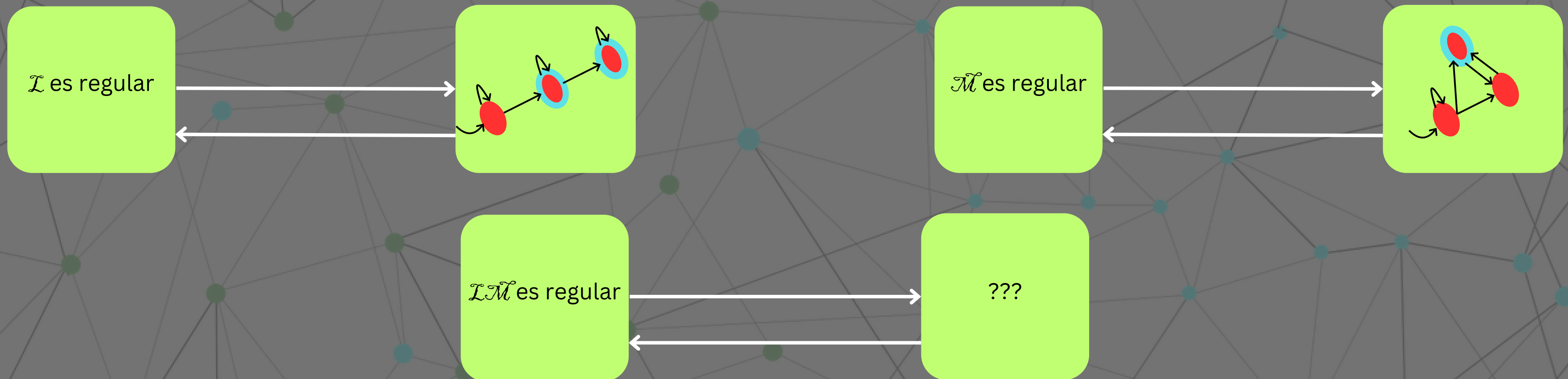


Cierre en Concatenación de Lenguajes Regulares

No se ve tan feo, suena parecido a la unión

Si $\mathcal{L}$ y $\mathcal{M}$ son regulares, entonces hay un autómata que los reconoce

y si $\mathcal{LM}$ es regular, entonces también hay un autómata que los reconoce



Cierre en Concatenación de Lenguajes Regulares

La concatenación es “pegar” una hilera después de la otra

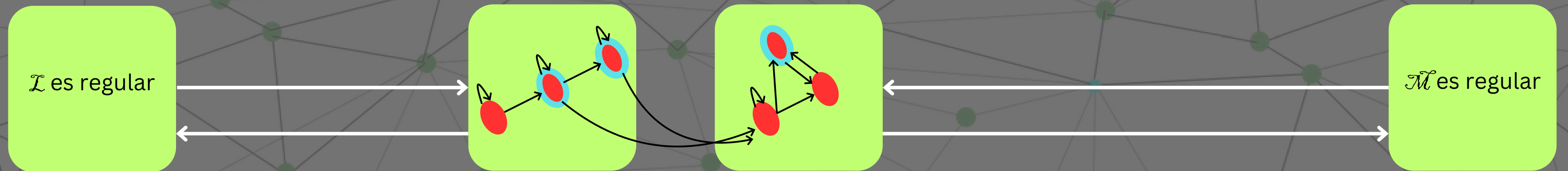
Tenemos un autómata que reconoce la primera parte de esta hilera (de $\mathcal{L}$), y otro que reconoce la otra parte (de M)

Necesitamos la aprobación de ambos autómatas



Cierre en Concatenación de Lenguajes Regulares

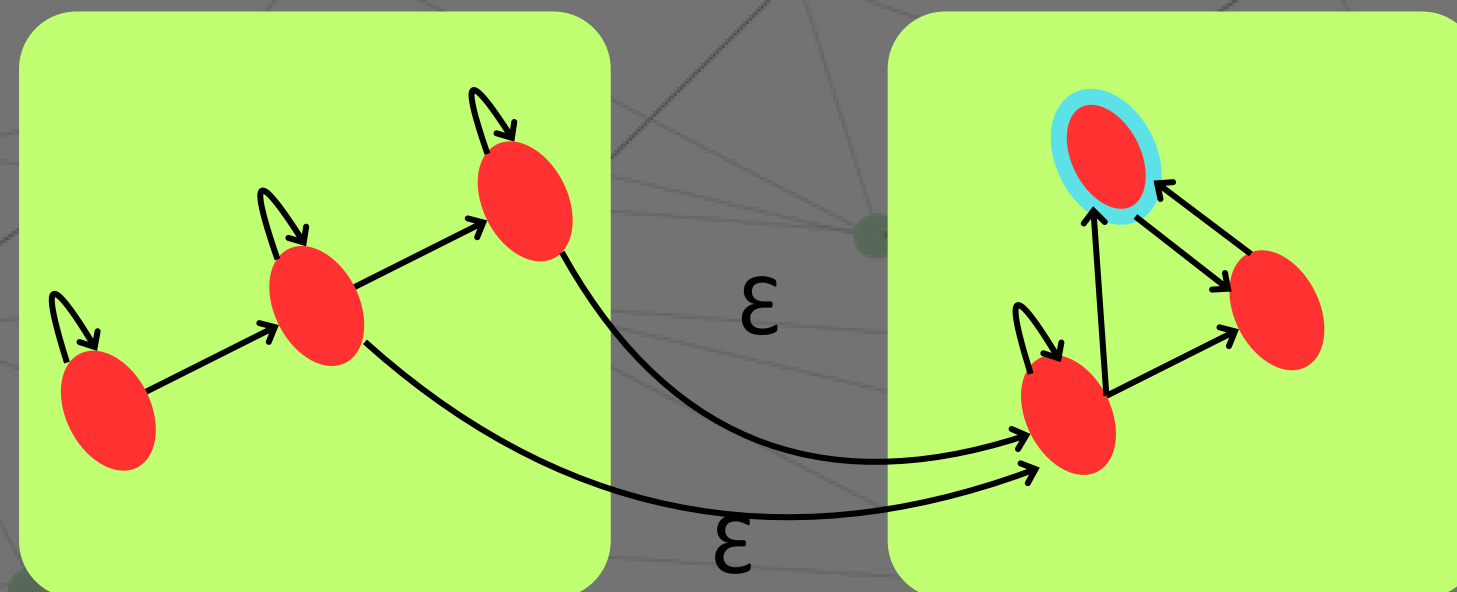
Entonces en principio podemos igual, así como en la unión, “unir” los dos autómatas, intentar “concatenar” ambos autómatas



Sabemos que si la hilera termina en un estado de aceptación, es porque es una hilera que pertenece al lenguaje, pero aquí tenemos una concatenación, es decir, hay otra hilera de otro lenguaje “pegada” entonces en lugar de aceptarla, nos vamos al otro autómatata para que haga su trabajo con esa otra parte, y de ahí ya lo aceptamos

Cierre en Concatenación de Lenguajes Regulares

Hacemos transición de los estados de aceptación, al inicial del otro autómata



Quitamos los estados de aceptación de los autómatas que no sean el último, para poner analizar todas las partes de la hilera

$\mathcal{LM}$ es regular

Flex tiene un autómata gigante que acepta la concatenación de todos las reglas/expresiones que le damos

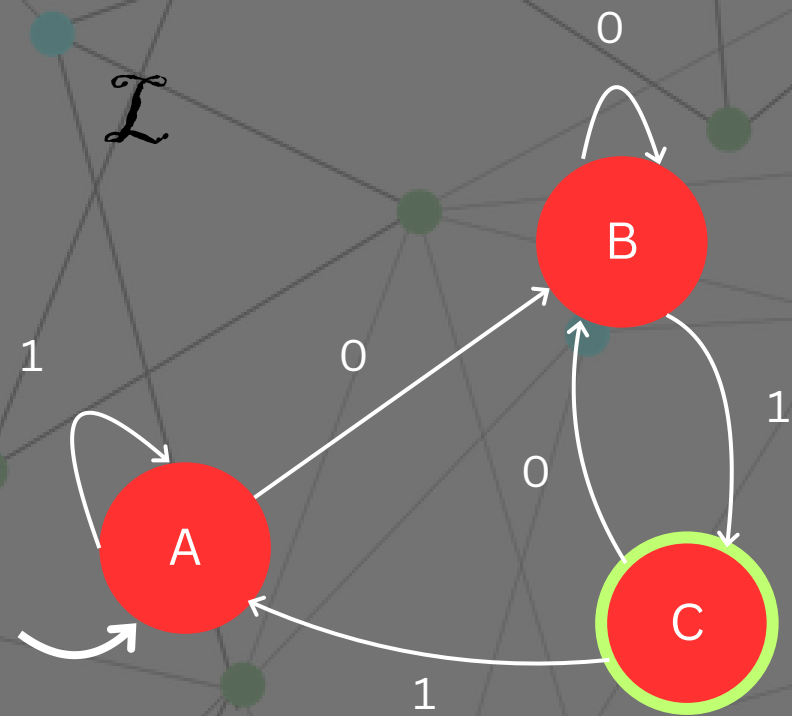
Cierre en Concatenación de Lenguajes Regulares

Ahora veamos un ejemplo

Tenemos dos lenguajes con sus respectivos autómatas

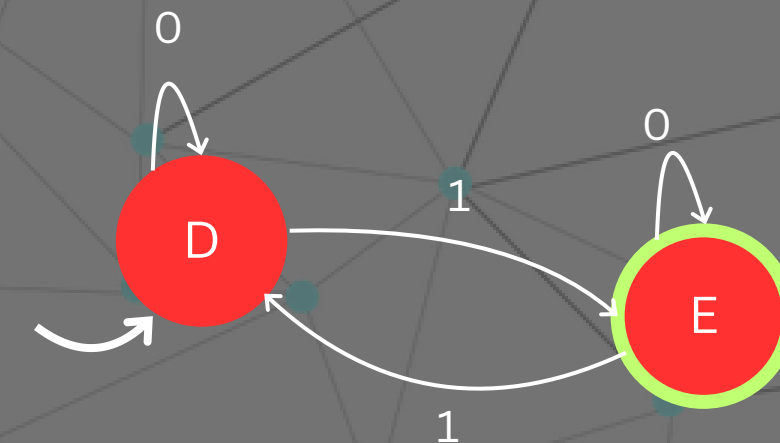
$\mathcal{L}$ = lenguaje sobre $\Sigma = \{0,1\}$ de hileras que terminen en 01

$\mathcal{M}$ = lenguaje sobre $\Sigma = \{0,1\}$ de hileras con un número impar de 1s



$\mathcal{L}\mathcal{M} = ???$

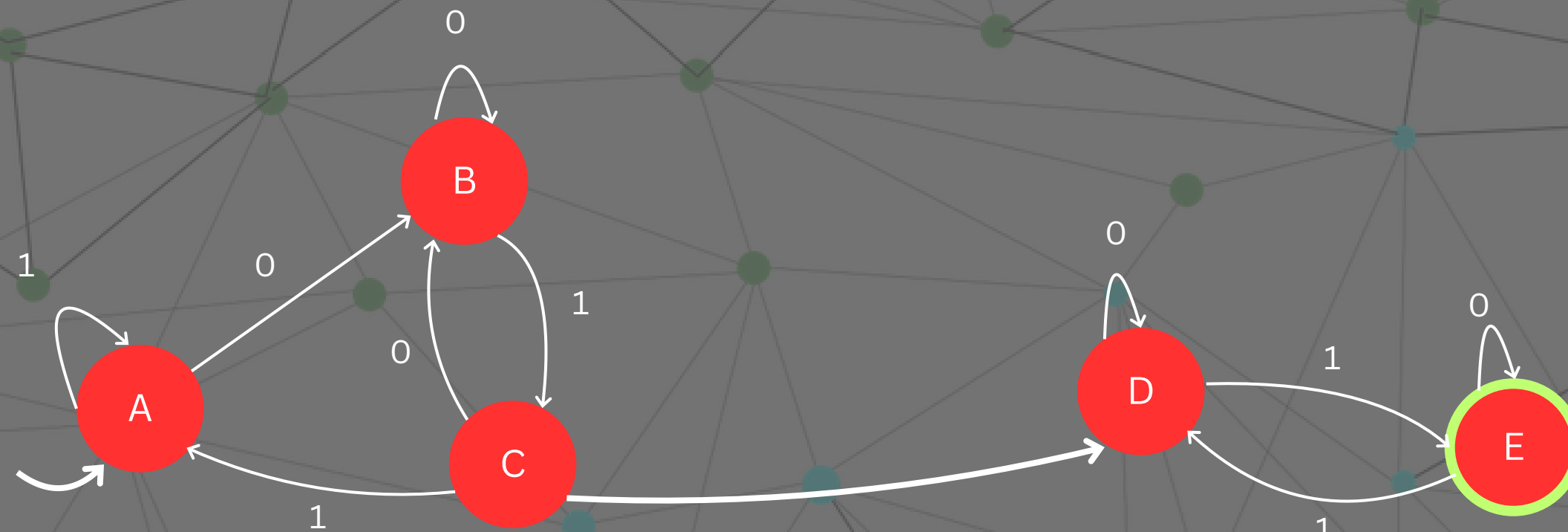
$\mathcal{M}$



Cierre en Concatenación de Lenguajes Regulares

$\mathcal{L}$ = lenguaje sobre $\Sigma = \{0,1\}$ de hileras que terminen en 01

$\mathcal{M}$ = lenguaje sobre $\Sigma = \{0,1\}$ de hileras con un número impar de 1s



$\mathcal{LM}$

Recordar que nos acaban de asignar 2 tareas cortas

Take a flower



¡Y eso fue
todo!